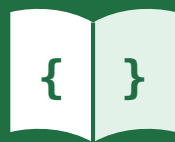

Module 20 — Docker and Deployment

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 An Image Is a Stack of Layers	2
2.2 Liveness and Readiness	2
2.3 The Twelve-Factor App	3
3. Step-by-Step Examples	3
3.1 The Application	3
3.2 A Multi-Stage Dockerfile	4
3.3 Buildpacks: An Image Without a Dockerfile	5
3.4 Health Probes and Graceful Shutdown	6
3.5 Configuration from the Environment	7
3.6 Docker Compose: Application and Database	8
4. Common Mistakes and Best Practices	9
5. Summary	9
6. Further Reading	10

1. Learning Goals

By the end of this module you will be able to:

- Write a multi-stage Dockerfile with a layered jar, a small JRE made with jlink, and a non-root user
- Build an image without a Dockerfile, with Cloud Native Buildpacks (spring-boot:build-image)
- Run the application and its database together with Docker Compose
- Configure liveness and readiness probes, and explain the difference
- Shut down gracefully, so that `docker stop` does not break running requests
- Configure one image for every environment with environment variables (12-factor)

Prerequisites: Module 15 (Actuator, health) · **Estimated time:** 4 hours · **Docker required**

2. Concepts

2.1 An Image Is a Stack of Layers

Every instruction in a Dockerfile (`COPY`, `RUN`) creates a layer. Docker caches layers and reuses them when nothing below them has changed. A Spring Boot fat jar is one file: change one line of code and the whole jar (60 MB of dependencies included) becomes a new layer. With a **layered jar**, the dependencies, the Boot loader and the application are separate layers. After a code change, only the small application layer is new.

Layer	Changes	Typical size
dependencies	when a library version changes	tens of MB
spring-boot-loader	when Boot is upgraded	< 1 MB
snapshot-dependencies	with every build of a SNAPSHOT library	small
application	with every code change	KB to a few MB

2.2 Liveness and Readiness

Probe	Question	When it fails, the platform ...	Should include
Liveness	Is the process broken beyond repair?	restarts the container	only the application's own state
Readiness	May it receive traffic now?	stops sending requests (no restart)	the dependencies it needs to serve requests

The classic mistake is a database check in liveness. When the database is down, every instance is restarted, which does not help the database and makes the outage worse.

2.3 The Twelve-Factor App

The “Twelve-Factor App” describes how to build applications that run well on platforms. Three of the factors matter most in this module:

- **III. Config:** configuration that differs between environments comes from the environment, not from the image.
- **IX. Disposability:** fast startup and graceful shutdown.
- **XI. Logs:** logs go to stdout, and the platform collects them (Boot does this by default).

3. Step-by-Step Examples

Run the application as usual first:

```
./mvnw -pl modules/20-docker-deployment/lesson spring-boot:run
```

```
=== readiness: ACCEPTING_TRAFFIC (shop "Bookstore") ===
probes: http://localhost:8080/actuator/health/liveness and /readiness
```

3.1 The Application

A small API on PostgreSQL (`/api/books`), the configuration and host name (`/api/shop`), and a slow endpoint for the shutdown example (`/api/slow?millis=...`). See requests.http.

i Note

SPEC decision 12: Paketo buildpacks have no Java 27 runtime yet. This module is therefore compiled with `maven.compiler.release=25`. The same jar runs on the JRE 27 of our Dockerfile and on the JRE 25 of the Buildpacks image.

3.2 A Multi-Stage Dockerfile

Build the jar first, then the image:

```
./mvnw -pl modules/20-docker-deployment/lesson package
docker build -t springbootedu/docker-deployment:jlink
↪ modules/20-docker-deployment/lesson
```

The first stage has a full JDK. It takes the jar apart into layers and builds a runtime with only the JDK modules the application needs:

```
# --- Stage 1: a full JDK 27 to take the jar apart and to build a runtime
FROM amazoncorretto:27-alpine AS builder
# jlink --strip-debug calls objcopy, which is part of binutils
RUN apk add --no-cache binutils
WORKDIR /builder
COPY target/*-SNAPSHOT.jar application.jar
# one folder per layer: dependencies change rarely, the application often
RUN java -Djarmode=tools -jar application.jar extract --layers --destination extracted
# which JDK modules does the application really use?
RUN jdeps --ignore-missing-deps --print-module-deps --recursive --multi-release 27 \
    --class-path 'extracted/dependencies/lib/*' extracted/application/application.jar
    ↪ > modules.txt
# a JRE with only these modules
RUN jlink --add-modules "${cat modules.txt}" \
    --strip-debug --no-man-pages --no-header-files --compress=zip-6 --output /jre
```

`jdeps` analyses the jars and prints the modules (here `java.base`, `java.sql`, `java.management`, ... — about 12 of the JDK's 70). `jlink` then builds a JRE with exactly these modules.

The second stage contains only this JRE and the layers:

```
# --- Stage 2: only the JRE and the application, on plain Alpine
FROM alpine:3.22
RUN addgroup -S spring && adduser -S spring -G spring
ENV JAVA_HOME=/opt/jre PATH="/opt/jre/bin:${PATH}"
COPY --from=builder /jre /opt/jre
WORKDIR /application
# the least changing layer first: a new build of the application reuses the cached
↪ dependency layer
```

```

COPY --from=builder /builder/extracted/dependencies/ ./
COPY --from=builder /builder/extracted/spring-boot-loader/ ./
COPY --from=builder /builder/extracted/snapshot-dependencies/ ./
COPY --from=builder /builder/extracted/application/ ./
USER spring
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=3 \
    CMD wget -qO- http://localhost:8080/actuator/health/liveness || exit 1
# exec form: java is PID 1 and receives SIGTERM from "docker stop" → graceful shutdown
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "application.jar"]

```

- **Non-root user:** if an attacker takes over the process, they are not root in the container.
- **Layer order:** the least changing layer first, so that a code change reuses the cached dependency layer.
- **Exec form** (["java", ...]): java is PID 1 and receives the SIGTERM of docker stop. With the shell form, a shell can be PID 1 and may not forward the signal.
- **-XX:MaxRAMPercentage=75:** the JVM sizes its heap from the container's memory limit, not from the host's memory.
- **HEALTHCHECK:** Docker (and Compose) show the container as healthy only when liveness answers.

.dockerignore sends only the jar to the Docker daemon, not the whole target/ folder.

image-sizes.sh builds three variants of the same application:

Image	Size
naive (Dockerfile.simple: full JDK, fat jar, root)	399 MB
Buildpacks (spring-boot:build-image)	353 MB
multi-stage + jlink (Dockerfile)	99 MB

★ Tip

On amazoncorretto:27-alpine, jlink --strip-debug needs objcopy. The builder stage installs it with apk add binutils. Without it, jlink fails with Cannot run program "objcopy".

3.3 Buildpacks: An Image Without a Dockerfile

Cloud Native Buildpacks detect a Java application, choose a JRE, create the layers and set good JVM defaults (memory calculator, non-root user). Spring Boot's Maven plugin runs them in Docker:

```
./mvnw -pl modules/20-docker-deployment/lesson spring-boot:build-image
```

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <name>springbootedu/docker-deployment:buildpacks</name>
      <env>
        <BP_JVM_VERSION>25</BP_JVM_VERSION>    <!-- no Java 27 in Paketo yet -->
      </env>
    </image>
  </configuration>
</plugin>
```

	Own Dockerfile	Buildpacks
Control	every line	through environment variables (BP_*)
Maintenance	you update base images	rebuild, and the builder brings the patches
Size	smallest with jlink	larger (full JRE, tools)
Java version	anything you can install	only what the buildpack offers

3.4 Health Probes and Graceful Shutdown

```
server:
  shutdown: graceful          # the default since Boot 3.4 – written here to
  ↪ make it visible
management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      probes:
        enabled: true         # /actuator/health/liveness and /readiness (the
  ↪ default in Boot 4)
      group:
```

```

    readiness:
      include: readinessState,db      # not ready without the database ...
    liveness:
      include: livenessState          # ... but still alive: a restart would not bring the
↪ database back

```

The application can also change its readiness itself, for example while it warms up a cache. `HealthProbesTest` shows that the application stays alive:

```

@Test
void anApplicationThatRefusesTrafficIsAliveButNotReady() {
    AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);    // e.g.
↪ during a long cache warm-up
    try {
        http.get().uri("/actuator/health/readiness").exchange()
            .expectStatus().isEqualTo(503)
            .expectBody().jsonPath("$.status").isEqualTo("OUT_OF_SERVICE");
        http.get().uri("/actuator/health/liveness").exchange().expectStatus().isOk();
    } finally {
        AvailabilityChangeEvent.publish(context, ReadinessState.ACCEPTING_TRAFFIC);
    }
}

```

Graceful shutdown: on `SIGTERM`, the web server stops accepting new requests, and running requests may finish. The limit is set here:

```

lifecycle:
  timeout-per-shutdown-phase: 20s      # how long running requests may still take after
↪ SIGTERM

```

Try it: call `/api/slow?millis=5000` and run `docker stop` while it runs. The request still returns 200 after 5 seconds, and then the container stops. `GracefulShutdownTest` proves the same without Docker: it closes the application context while a request runs. The platform must wait longer than this timeout before it kills the process (`stop_grace_period` in Compose, `terminationGracePeriodSeconds` in Kubernetes, module 22).

3.5 Configuration from the Environment

```

@ConfigurationProperties("bookstore.shop")
public record ShopProperties(@DefaultValue("Bookstore") String name,
↪ @DefaultValue("TRY") String currency) {
}

```

Boot's **relaxed binding** turns environment variables into properties: `BOOKSTORE_SHOP_NAME` → `bookstore.shop.name`, `SPRING_DATASOURCE_URL` → `spring.datasource.url`. The same image runs in every environment. Only the environment variables differ. `ShopPropertiesTest` binds such variables without starting the application.

! Warning

Secrets (passwords, tokens) come from the environment or a secret store, never from `application.yaml` in the image. `compose.yaml` uses `${POSTGRES_PASSWORD:-bookstore}`: the default is only for local development.

3.6 Docker Compose: Application and Database

```
services:
  postgres:
    image: pgvector/pgvector:0.8.6-pg18
    environment:
      POSTGRES_DB: bookstore
      POSTGRES_USER: bookstore
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-bookstore} # dev default only; set it in
↪ the environment
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bookstore -d bookstore"]
      interval: 5s
      timeout: 3s
      retries: 10
      # no "ports": only the app can reach the database, over the compose network

  app:
    build: lesson # lesson/Dockerfile
    image: springbootedu/docker-deployment:jlink
    ports:
      - "8080:8080"
    environment: # 12-factor: config from the
↪ environment
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/bookstore
      SPRING_DATASOURCE_USERNAME: bookstore
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD:-bookstore}
      BOOKSTORE_SHOP_NAME: ${BOOKSTORE_SHOP_NAME:-Bookstore on Compose}
    depends_on:
      postgres:
        condition: service_healthy # start only when the database
↪ answers
    stop_grace_period: 30s # longer than
↪ spring.lifecycle.timeout-per-shutdown-phase
```

```
./mvnw -pl modules/20-docker-deployment/lesson package
docker compose -f modules/20-docker-deployment/compose.yaml up --build
```

- The services find each other by name on the compose network (postgres:5432).
- The database publishes no port: only the application can reach it.
- `depends_on` with condition: `service_healthy` starts the application only when `pg_isready` answers.

DockerComposeIT runs after package: it starts this compose file with Testcontainers (ComposeContainer), which also builds the Dockerfile, waits for readiness and calls the API.

4. Common Mistakes and Best Practices

!! Caution

Never put the database (or another dependency) into the liveness probe. A database outage would restart every instance in a loop.

- **Do:** use the exec form for ENTRYPOINT, and give the platform a grace period longer than `spring.lifecycle.timeout-per-shutdown-phase`.
- **Don't:** run containers as root, or put build tools (a JDK, Maven) into the runtime image.
- **Do:** order the layers from rarely to often changing, and keep the build context small with `.dockerignore`.
- **Don't:** bake environment-specific values or secrets into the image. Pass them as environment variables.
- **Do:** pin base image versions (alpine:3.22, amazoncorretto:27-alpine) and rebuild regularly for security patches.
- **Don't:** use `depends_on` without a condition. It waits only until the container has started, not until the database answers.

5. Summary

- A multi-stage Dockerfile separates building from running: layered jar, a JRE made with `jdeps` + `jlink`, a non-root user. Here: 99 MB instead of 399 MB.
- Buildpacks build a good image without a Dockerfile, with less control over size and Java version.
- Liveness asks “restart me?”, readiness asks “send me traffic?”. Dependencies belong only in readiness.
- Graceful shutdown lets running requests finish after SIGTERM, within `timeout-per-shutdown-phase`.

- One image for all environments: configuration from environment variables through relaxed binding.
- Docker Compose runs the application and its dependencies together, started in the right order with health checks.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Dockerfiles](#) · [Cloud Native Buildpacks](#)
- [Spring Boot — Graceful Shutdown](#)
- [Spring Boot — Application Availability](#)
- [Docker — Multi-stage builds](#) · [Compose startup order](#)
- [The Twelve-Factor App](#)